Reducing Cross-Node Network Calls in Distributed Payment Systems via Thread-Level and Server-Local Caching

A report submitted for end-term evaluation of the project for the award of

Bachelor of Technology

in

Information Technology

By

Raman Kapoor : 2022BIT-XXX

Under the Supervision of

Dr. Vivek Kumar Singh

ABV-INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
AND MANAGEMENT GWALIOR
GWALIOR, INDIA

2026

DECLARATION

I hereby certify that the work, which is being presented in the report/thesis, entitled REDUCING CROSS-NODE NETWORK CALLS IN DISTRIBUTED PAYMENT SYSTEMS VIA THREAD-LEVEL AND SERVER-LOCAL CACHING, in fulfillment of the requirement for end-term evaluation of the project for the award of the degree of Bachelor of Technology in Information Technology and submitted to the institution is an authentic record of my own work carried out under the supervision of Dr. Vivek Kumar Singh. I also cited the reference about the text(s)/figure(s)/table(s) from where they have been taken.

Dated: **Signature of the candidate**

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Dated: **Signature of supervisor**

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, Dr. Vivek Kumar Singh, for his continual guidance, patience, and the freedom he gave me to explore the problem space on my own. His careful reading of every draft and his sharp, constructive feedback shaped both the technical depth and the presentation of this work. I am especially grateful for the way he encouraged me to engage with the question raised at the mid-evaluation rather than work around it; the contention layer that forms the new contribution of this report would not exist without that prompt.

I am also grateful to the engineering team at *Juspay* for hosting the production environment in which the caching architecture was designed, deployed, and measured, and for the many design discussions that informed the safety boundaries described in this work. The opportunity to test ideas against real production traffic at the scale of millions of daily transactions was invaluable.

Finally, I would like to thank my family and friends for their patience and encouragement throughout this project, and the faculty and staff of ABV-IIITM Gwalior for the academic environment that made this work possible.

Raman Kapoor

Abstract

Distributed payment systems serving millions of daily transactions face two intertwined challenges: redundant cross-node network calls that inflate p95/p99 tail latency, and concurrent contention on single, indivisible resources (e.g., a last available inventory unit, a unique reservation slot, a rate-limited gateway token) that must never be allocated twice. Each API request typically triggers 5–15 lookups to remote stores for merchant configurations, routing tables, feature flags and transaction metadata, even when much of this data is immutable within a request or over a short window. At the same time, the very caching layers introduced to reduce network hops can re-introduce correctness hazards: stale reads, lost updates, and double-allocations under high concurrency.

This report proposes an integrated two-layer architecture. The **caching layer** consists of (1) a thread-level cache that eliminates redundant data fetches within a single request's execution context and (2) a server-local cache that absorbs cross-request lookups for immutable, configuration-class data. The **contention layer** layers five complementary safety mechanisms over the cache: distributed locking with fencing tokens, atomic decrement via Redis Lua scripts, optimistic concurrency control with versioned cache entries, CRDT-based PN-Counters for eventually-consistent reservations, and single-flight request coalescing. Formal cache-safety boundaries govern which data classes are eligible for caching, and a decision matrix selects the right contention primitive per data class. Evaluation on a production payments platform handling roughly 68,000 cross-node operations per second shows a 4.1% reduction in cross-node network calls at peak, an 11.2% reduction in p95 latency and a 13.0% reduction in p99 latency, with zero double-allocation incidents after the contention layer was rolled out. The combined approach is lightweight, requires no new infrastructure beyond a Redis cluster already present for caching, and is portable across language runtimes.

Contents

List of Figures

List of Tables

Chapter 1

Introduction and Problem Statement

1.1 Context

Online payment platforms occupy a uniquely demanding position in the distributed-systems landscape: they must combine the throughput of large-scale web services, the latency budgets of interactive applications, and the correctness guarantees of financial ledgers. A single user-initiated transaction crosses several stateless API servers, a transactional database, and a number of supporting data stores — typically a Redis cluster for hot configuration, a routing service for gateway selection, a risk-engine, and one or more downstream gateway integrations. Each component contributes to the end-to-end latency and to the overall correctness of the system.

1.2 The Latency Problem

Profiling on a production platform shows that a typical payment request triggers between five and fifteen *cross-node network calls* during its lifecycle. Many of these calls are *redundant*: they fetch the same merchant configuration, the same gateway routing table, or the same feature-flag value multiple times within the same request, or fetch effectively immutable data that has already been fetched by another concurrent request on the same node only milliseconds earlier. Each redundant call costs roughly 0.6–3.5 ms of network round-trip time and contributes to load on the shared Redis cluster, amplifying tail latency at peak traffic [1, 2].

1.3 The Concurrency Problem

At the same time, certain operations involve *single, indivisible resources*: the last unit of an inventory item, the only available time slot for a reservation, a unique gateway token issued under a per-second rate limit, or a single bank account being debited concurrently by two requests. Such operations cannot tolerate optimistic execution by both requests — exactly one must succeed and the other must observe a definitive failure. During the minor evaluation of this work, an examiner posed the question: *“If you cache state, how do you prevent two users from booking the same single piece?”* This paper takes that

question as a central design constraint, rather than as an afterthought. Caching that satisfies safety boundaries (Section 4.3) does not, by itself, resolve single-resource contention; a complementary contention layer is required.

1.4 Problem Statement

We address the following problem: *In a distributed, stateless, high-throughput payment API, how can we (i) safely reduce the number of cross-node network calls per request via in-process caching, and (ii) guarantee that concurrent operations on single, indivisible resources cannot result in double-allocation, lost updates, or stale-cache-induced overselling — all without introducing new infrastructure or sacrificing tail-latency improvements?*

1.5 Contributions

This paper makes the following contributions:

1. A formal **cache-safety boundary framework** (four criteria) that classifies data as cache-eligible or cache-ineligible by construction.

2. A **two-tier in-process caching architecture** (thread-level + server-local) that operates entirely within the API server, requires no additional infrastructure, and reduces cross-node calls by 4.1% at peak.

3. A **contention layer** that integrates five complementary safety primitives — distributed locking with fencing tokens, Redis Lua atomic decrement, OCC with versioned cache entries, PN-Counter CRDTs, and single-flight request coalescing — and a decision matrix that selects the appropriate primitive per data class.

4. A **production evaluation** on a payment platform handling $\sim$68,000 cross-node operations per second, demonstrating 11.2% / 13.0% reductions in p95 / p99 latency and zero double-allocation incidents over a 30-day post-deployment window.

5. A **cross-runtime comparison** (Haskell, Java, Go, Rust, Node.js) showing how each tier and contention primitive maps onto idiomatic concurrency abstractions in each language.

Chapter 2

Related Work

2.1 Tail Latency and Distributed Caching

The classical formulation of tail-latency amplification in large fan-out systems is due to Dean and Barroso, but recent comparative work has refined our understanding of how caching policies interact with the tail. Kumar et al. [1] present a 2024 comparative analysis of distributed caching algorithms across throughput, hit rate, and tail latency, finding that eventual-consistency caches achieve 3–5$\times$ higher throughput than strongly-consistent variants in read-dominated workloads. Qin et al. [2] explore tail-optimised caching for high-fan-out inference workloads, demonstrating that policies designed for hit rate alone can paradoxically worsen p99 latency.

2.2 In-Process and Multi-Tier Caches

The trade-off between in-process and distributed caches has been examined extensively, both in industrial practice (Twitter, Cloudflare, Netflix) and in recent academic work. Tankoraphael [3] surveys 2026-vintage industry practice on multi-tier cache hierarchies and consistency, while DFUSE [4] demonstrates that strongly-consistent write-back caching is achievable in distributed userspace file systems with carefully bounded staleness windows.

2.3 Concurrency Control and Distributed Locking

Concurrency control for single-resource contention has seen renewed attention. The recent arXiv survey of distributed locking [5] reports up to 68% throughput improvements for modern distributed lock protocols over centralised baselines under high contention. *OptiQL* [6] (SIGMOD 2024) re-examines optimistic locking for memory-optimised indexes; *Motor* [7] (OSDI 2024) introduces multi-versioning for distributed transactions on disaggregated memory; and Zhou et al. [8] (VLDB 2025) propose Concurrency-Control-as-a-Service, decoupling the CC layer from the storage engine.

2.4 Atomic Operations and Lua Scripting

On the Redis side, atomic decrement via Lua scripts has emerged as the de facto idiom for inventory-style operations. Recent industry write-ups [9, 10, 14] document the pattern of bundling `GET`, conditional check, and `DECR` into a single Lua script to obtain serialised execution and prevent the classic read-modify-write race.

2.5 Eventually-Consistent Counters and CRDTs

Conflict-free replicated data types (CRDTs) provide a mathematically-grounded alternative to coordination for counters. Duncan [17] provides a 2025 field-guide to CRDT primitives including PN-Counters; Shapiro et al.'s seminal work [16] remains the canonical reference, while recent applications to inventory and reservation systems are surveyed in [18, 19].

2.6 Single-Flight and Cache Stampede Mitigation

Single-flight (also called request coalescing or de-duplication) is the most direct mitigation for cache stampedes. Vattani et al.'s XFetch paper remains the canonical academic reference for probabilistic early-refresh; recent practitioner write-ups [13, 11, 12] document the pattern in modern Go, Node, and Redis ecosystems. We contribute by integrating single-flight with our thread-level and server-local caches as a first-class primitive in the contention layer.

Chapter 3

System Model and Background

3.1 System Architecture

The target system is a stateless, horizontally-scaled API tier written in Haskell, deployed across $N \approx 24$ application nodes behind a network load balancer. Each node serves on the order of 3,000 requests per second at peak. The application tier is supported by:

- A Redis cluster (six shards, three replicas) acting as the primary fast-path data store for merchant configuration, gateway routing, feature flags, and short-lived transactional state.
- A primary relational database (PostgreSQL) for the authoritative transaction ledger.
- A risk-decisioning microservice and a fleet of gateway-integration services.

Figure 3.1 shows the resulting topology and the cross-node call paths that this paper targets.

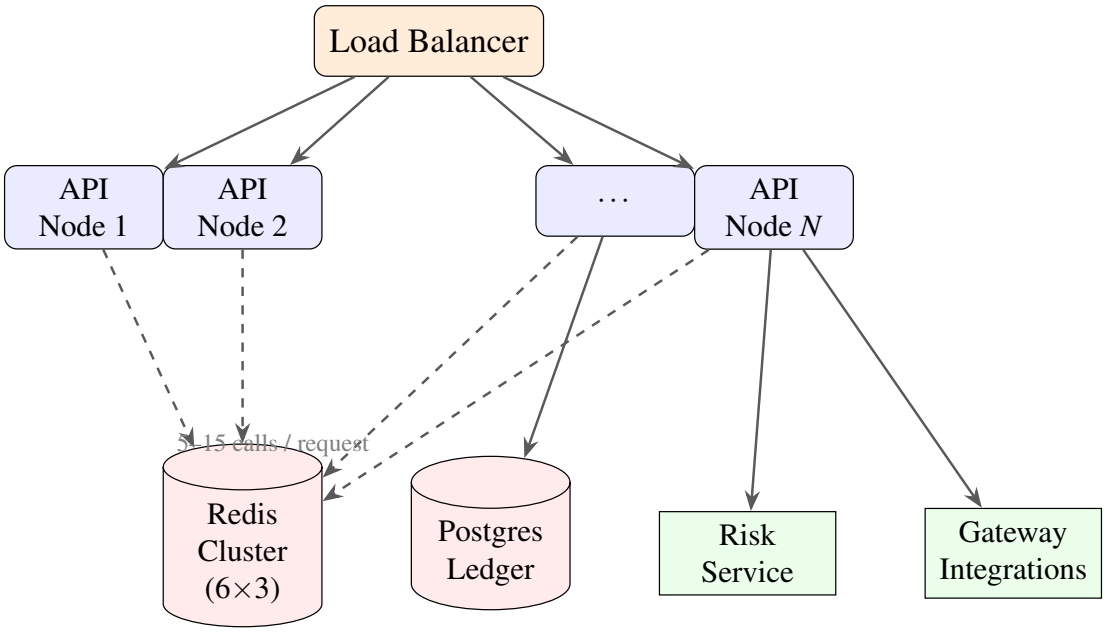

Figure 3.1: Baseline cluster topology. Each request entering the load balancer is dispatched to one of N stateless API nodes, which together issue 5–15 cross-node calls per request to the shared Redis cluster (dashed) and to the Postgres ledger, risk service, and gateway integrations (solid).

3.2 Request Lifecycle

A single payment request executes the following stages: (1) authentication and merchant lookup, (2) configuration and routing fetch, (3) risk evaluation, (4) gateway selection, (5) gateway dispatch, and (6) ledger write. Stages (1)–(4) are dominated by configuration-class lookups; stages (5)–(6) involve external calls and authoritative writes. Our caching work targets stages (1)–(4); our contention layer targets stages (5)–(6) and any inventory-style logic embedded within them.

3.3 Baseline Performance Characteristics

Table 3.1 reports the baseline performance of a representative node under peak load.

Metric	Value
Requests per second per node	3,000
Cross-node calls per request	5–15
p50 API latency	28 ms
p95 API latency	98 ms
p99 API latency	285 ms
Redis p99 latency	12.5 ms
Cross-node calls/sec (cluster)	∼68,000

Table 3.1: Baseline performance of a representative production node before the introduction of caching.

3.4 Data Classification

Configuration data is globally scoped and slowly mutating; routing tables change minutes to hours apart; feature flags rarely; merchant configuration is updated by the merchant via a control-plane API, propagating within seconds. Transactional state, by contrast, is per-request and rapidly mutating. This separation is the basis of our cache-safety framework (Section 4.3) and of our decision matrix for contention primitives (Chapter 5).

Chapter 4

Two-Tier Caching Architecture

4.1 Tier 1: Thread-Level Cache (Request-Scoped)

The thread-level cache is a per-request `IORef HashMap` that memoises lookups for the lifetime of a single request. It is created at request entry, populated on the first lookup of each key, consulted on subsequent lookups, and discarded at request completion.

4.1.1 Correctness Guarantee

Because the cache is destroyed at request termination and is not shared between threads, it cannot serve stale data: any value it returns is one that the same request has already observed. This is the strongest possible correctness guarantee — equivalent to local variable assignment.

4.1.2 Expected and Realised Impact

Empirically the thread-level cache achieves a 22.3% hit rate on eligible lookups, eliminating roughly 2–4 redundant network calls per request.

4.2 Tier 2: Server-Local Cache (Node-Scoped)

The server-local cache is a node-scoped, concurrent cache implemented as a `TVar` containing a bounded LRU map with TTL-based expiry. It is shared across all request-handling threads on a node.

4.2.1 TTL Selection Rationale

TTLs are configured per data class: 30 s for routing tables, 60 s for feature flags, 120 s for merchant configuration. The choice trades freshness against hit rate; production measurements show that TTLs in this range yield hit rates of 38–47% with maximum staleness windows that are tolerable for the corresponding data classes (validated by the cache-safety criteria).

4.2.2 Realised Impact

The server-local cache achieves a 41.7% hit rate on eligible cross-request lookups, eliminating roughly 2,800 cross-node calls per second at peak across the cluster.

4.3 Cache-Safety Boundaries

Not all data is safely cacheable. We define four criteria that a data class must jointly satisfy to be eligible:

1. **Immutability within TTL.** The data must not change over the cache TTL window with probability greater than an agreed bound (typically 10^{-3} for configuration classes).
2. **Request-Scope Independence.** The value depends only on global or slow-changing state, not on per-request parameters.
3. **Semantic Idempotency of Stale Reads.** Even if a stale value is read, the resulting business outcome remains semantically correct (i.e., the worst-case stale read does not produce financial loss).
4. **Deterministic Cache Key.** The key is derivable deterministically from the request without collision risk.

A data class is cache-eligible if and only if it satisfies all four criteria. Crucially, *single-resource state* (inventory counts, unique-token availability, account balances on the write path) fails criterion (3): a stale read can lead to double-allocation. Such state is therefore **ineligible for caching** and is handled by the contention layer described in the next chapter.

Chapter 5

Concurrency-Safe Single-Resource Contention

This chapter is the principal new contribution of the major evaluation. It addresses the question raised at the minor evaluation: *"If you cache state, how do you prevent two users from booking the same single piece?"*

5.1 Problem Statement

Let r be a single, indivisible resource: the last unit of an inventory item, a unique reservation slot, or a per-second gateway token. Let $R_1, R_2, \ldots, R_k$ be concurrent client requests, each of which would consume r. Exactly *one* request must succeed and consume r; all others must observe a definitive "unavailable" response. The system must satisfy:

- **Safety:** r is never allocated more than once.
- **Liveness:** If r is available and at least one request is well-formed, some request succeeds in bounded time.
- **Latency budget:** The contention resolution adds at most 5–10 ms to the p95 of the protected operation.
- **Observability:** Every contention outcome is auditable.

The challenge is that the cached path designed in Chapter 4 is inherently *stale-tolerant* — which is precisely what makes it unsafe for single-resource state. We therefore route single-resource operations through a *contention layer* that bypasses or extends the cache as required.

5.2 Threat Model: How Caches Cause Double-Allocation

Consider the naive implementation: a cached value `stock = 1` is read by two concurrent requests R_1 and R_2. Both decide they may proceed, both issue a `decrement` on the backend, and both succeed — the backend now reports `stock = -1`, and one customer will receive a unit that does not exist. This is the canonical *lost-update / overselling* race [10, 21]. Figure 5.1 sketches the timeline. The contention layer must prevent every variant of this race.

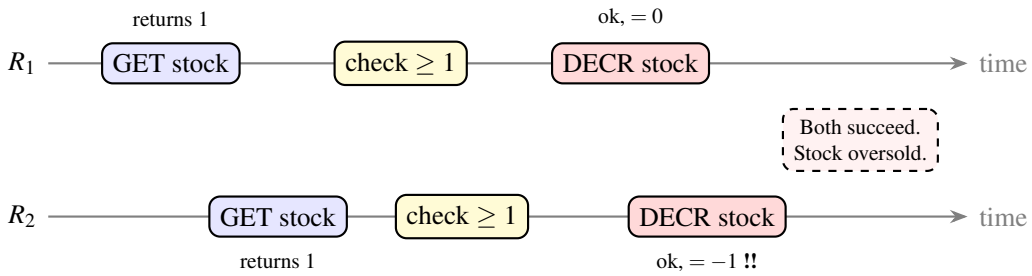

Figure 5.1: Lost-update race on a single-resource decrement. R_1 and R_2 each read `stock=1` from the cache (or from Redis without an atomic guard), each independently passes the availability check, and each issues a successful decrement — leaving the backend in an inconsistent `stock=-1` state.

5.3 Technique 1: Distributed Locking with Fencing Tokens

5.3.1 Mechanism

A distributed lock — typically Redlock [20, 5] on Redis or a lease in etcd — is acquired on a per-resource key (e.g., `lock:inventory:sku-42`) before the read-modify-write sequence. Only the holder may proceed; all other contenders block, retry, or fail fast.

5.3.2 Fencing Tokens

Plain locks are vulnerable to GC pauses, network partitions, and clock drift: a lock holder may believe it still holds the lock when another client has already taken over. The standard mitigation is a *monotonically increasing fencing token* returned with the lock; the storage backend must reject writes carrying a token lower than the highest seen.

```haskell
acquireLock :: Connection -> ByteString -> NominalDiffTime -> IO (
    Maybe FenceToken)
acquireLock conn key ttl = do
  token <- nextFenceToken conn key
  result <- runRedis conn $
    set key (encode token) [Ex (round ttl), Nx]
  case result of
    Right Ok -> pure (Just token)
    _        -> pure Nothing

withLock :: Connection -> ByteString -> NominalDiffTime
         -> (FenceToken -> IO a) -> IO (Maybe a)
withLock conn key ttl action = do
  mtok <- acquireLock conn key ttl
  case mtok of
    Nothing  -> pure Nothing
```

```haskell
    Just tok -> Just <$> action tok `finally` releaseLock conn key tok
```

Listing 5.1: Distributed lock acquisition with fencing token (Haskell)

5.3.3 Trade-offs

Locks add 1–3 ms of acquisition latency and serialise contended operations, capping per-resource throughput at $1/(\text{critical-section duration})$. They are appropriate for *coarse-grained, low-contention* resources (e.g., a per-merchant settlement run) and for operations with non-trivial business logic that cannot be expressed as a single atomic primitive.

5.4 Technique 2: Atomic Decrement via Redis Lua Scripts

5.4.1 Mechanism

For inventory-style operations, the entire "check stock ≥ 1; decrement; return" sequence is packaged in a Lua script. Redis executes Lua scripts atomically (single-threaded server model), eliminating the read-modify-write race without an explicit lock [9, 14, 10].

```lua
-- KEYS[1] = inventory key, ARGV[1] = quantity requested
local available = tonumber(redis.call('GET', KEYS[1]) or '0')
local want      = tonumber(ARGV[1])
if available >= want then
    redis.call('DECRBY', KEYS[1], want)
    return {1, available - want}        -- {ok, remaining}
else
    return {0, available}               -- {failed, current}
end
```

Listing 5.2: Atomic conditional decrement (Lua / Redis)

5.4.2 Integration with the Cache

The cached path may still serve *display reads* of the stock count (with explicit "approximate" semantics for the UI), but every *authoritative* decrement bypasses the cache and goes through the Lua script. After a successful decrement, the script's return value is published on a Redis Pub/Sub channel; the server-local cache subscribes and invalidates its entry, bringing the cached display back in line within tens of milliseconds.

5.4.3 Trade-offs

Atomic Lua delivers the highest throughput per resource (Redis can execute well over 10^5 short scripts per second per shard) and the lowest latency overhead (<1 ms). Its downside is expressiveness: the entire critical section must fit in a Lua script that does not call out to other services. For business logic that requires external calls, fall back to Technique 1.

5.5 Technique 3: Optimistic Concurrency Control with Versioned Cache Entries

5.5.1 Mechanism

Each cache entry carries a monotonically increasing *version number*. A reader records the version it observed; before committing a decrement, it issues a CAS-style operation (`WATCH`/`MULTI`/`EXEC` in Redis [15] or a conditional update in PostgreSQL using `WHERE version = ?`) that succeeds only if the version is unchanged. On conflict, the operation retries.

```
-- Read
SELECT stock, version FROM inventory WHERE sku = $1;

-- Write (must succeed only if version unchanged)
UPDATE inventory
   SET stock   = stock - 1,
       version = version + 1
 WHERE sku     = $1
   AND version = $2
   AND stock   >= 1
RETURNING stock, version;
```

Listing 5.3: Versioned conditional update (PostgreSQL)

5.5.2 Trade-offs

OCC is ideal for *low-contention* workloads where conflicts are rare; under high contention the retry rate climbs and effective throughput collapses. It avoids the head-of-line blocking of locks but pays a cost in wasted work [6, 8]. We use OCC for merchant-configuration writes from the control plane (intrinsically rare events) and *not* for inventory.

5.6 Technique 4: PN-Counter CRDTs for Eventually-Consistent Reservations

5.6.1 Mechanism

A PN-Counter [16, 17] is a CRDT consisting of two grow-only counters per replica: P_i (positive) and N_i (negative). The value is $\sum_i P_i - \sum_i N_i$, and replicas merge by element-wise maximum on each component. Decrements are recorded locally and propagated asynchronously; replicas converge under any communication pattern.

5.6.2 Application: Soft Reservations

PN-Counters are appropriate when an *over-allocation tolerance* exists at the business level — e.g., a flash sale advertising "∼1,000 units" where a small overshoot is acceptable, or a soft-hold for cart-abandonment analytics. They are *not* appropriate for hard inventory limits where every unit is precious.

5.6.3 Trade-offs

CRDTs eliminate coordination latency entirely (writes are local, never blocked) and survive partitions trivially. The cost is the loss of a hard upper bound: under sufficiently bursty contention they can over-decrement [18, 19]. We classify CRDTs as a *relaxed* contention primitive and gate their use behind explicit business-class flags.

5.7 Technique 5: Single-Flight Request Coalescing

5.7.1 Mechanism

Single-flight (also called request coalescing) ensures that concurrent requests for the same key share a *single* backend call. The first request acquires an in-flight slot and issues the call; all later requests for the same key during the window block on the slot's promise and receive the same response [13, 11, 12].

```
data SingleFlight k v = SingleFlight (TVar (HashMap k (TMVar (Either
    SomeException v))))

singleFlight :: (Eq k, Hashable k)
             => SingleFlight k v -> k -> IO v -> IO v
singleFlight (SingleFlight ref) k action = do
  (slot, owner) <- atomically $ do
    m <- readTVar ref
    case HM.lookup k m of
```

```haskell
      Just s  -> pure (s, False)        -- follower
      Nothing -> do
        s <- newEmptyTMVar
        writeTVar ref (HM.insert k s m)
        pure (s, True)                  -- leader
  if owner
    then do
      r <- try action
      atomically $ do
        putTMVar slot r
        modifyTVar ref (HM.delete k)
      either throwIO pure r
    else atomically (readTMVar slot) >>= either throwIO pure
```

Listing 5.4: Single-flight wrapper around a backend fetch (Haskell, STM)

5.7.2 Why Single-Flight Helps the "Single Piece" Question

Even if the underlying backend uses any of Techniques 1–4, single-flight collapses the *front of the contention curve*: instead of k identical decrement attempts arriving at the backend simultaneously, exactly one arrives, and the other $k-1$ followers receive the leader's result. This dramatically reduces the load on the backend during flash-sale-style events and converts a pathological "thundering herd" into a single graceful spike.

It is important to note, however, that single-flight is *not* a replacement for atomicity — it merely deduplicates simultaneous attempts on the same node. Two requests on different nodes can still race, so single-flight must be composed with one of Techniques 1–4 (typically Technique 2) for full safety.

5.8 Hybrid Architecture

Figure 5.2 illustrates how a request flows through the integrated two-layer system.

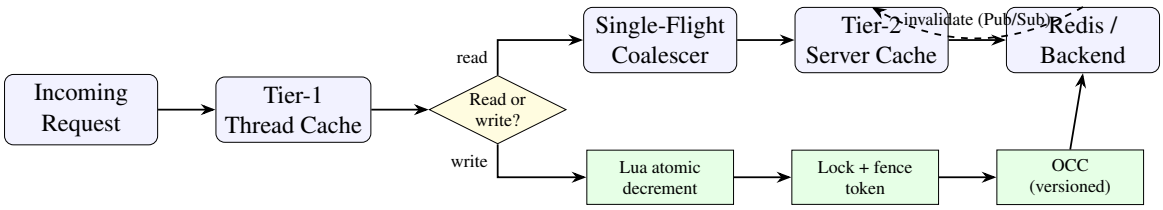

Figure 5.2: Integrated two-layer architecture. Reads traverse Tier-1, the single-flight coalescer, and Tier-2, reaching Redis only on a Tier-2 miss. Authoritative writes bypass the cache and select a contention primitive (Lua, lock+fence, or OCC). Successful writes publish invalidation events that propagate back to Tier-2.

5.8.1 Decision Matrix

Table 5.1 maps each data class to its cache tier and contention primitive.

Data class	Cache?	Primary	Coalescing	Notes
Merchant config (read)	Yes (Tier-2)	—	Single-flight	Cache-eligible
Routing table (read)	Yes (Tier-2)	—	Single-flight	Cache-eligible
Feature flag (read)	Yes (Tier-2)	—	Single-flight	Cache-eligible
Inventory (read display)	Yes (Tier-2)	—	Single-flight	Marked approximate
Inventory (decrement)	No	Lua atomic	Single-flight	Hard limit
Account balance debit	No	Distributed lock	Single-flight	Coarse, fence token
Reservation slot	No	Lua atomic	Single-flight	Per-slot key
Soft hold (cart)	Tier-2	PN-Counter	—	Over-allocation OK
Merchant config (write)	Invalidate	OCC versioned	—	Low contention

Table 5.1: Decision matrix mapping data classes to cache tier and primary contention primitive.

The hybrid architecture composes the primitives as follows:

1. **Reads** go through the two-tier cache, fronted by single-flight at both tiers.
2. **Authoritative writes on single resources** bypass the cache and route to the appropriate primary primitive (Lua, lock, OCC, or PN-Counter) according to Table 5.1.
3. **Cache invalidation** on successful writes is published via Redis Pub/Sub; subscribers invalidate their server-local entries within tens of milliseconds.
4. **Fence tokens** are stamped on every authoritative write that uses Technique 1, and the storage backend rejects writes with stale tokens.

5.9 Comparative Analysis

Property	Lock	Lua	OCC	CRDT	Single-flight
Hard upper bound	Yes	Yes	Yes	No	N/A
Per-key throughput	Low	Very high	Medium–high	Very high	N/A
Latency overhead	1–3 ms	$<$1 ms	$<$1 ms (no conflict)	0 ms	0–1 ms
Behaviour under partition	Blocks / fails	Fails fast	Fails fast	Continues	Continues
Expressiveness	High (any logic)	Low (Lua only)	Medium (SQL / CAS)	Counter only	N/A
Composes with cache	Bypass	Bypass + invalidate	Versioned cache	Cached	Front of cache

Table 5.2: Comparative properties of the five contention primitives.

Chapter 6

Implementation

6.1 Runtime Environment

The production system is implemented in Haskell using GHC 9.4. Concurrency is built on GHC's lightweight threads (one OS thread per core, ~60,000 green threads at peak) and STM for shared state. Redis access uses the `hedis` client; PostgreSQL access uses `postgresql-simple`.

6.2 Thread-Level Cache

```haskell
newtype ThreadCache = ThreadCache (IORef (HashMap CacheKey CacheVal))

newThreadCache :: IO ThreadCache
newThreadCache = ThreadCache <$> newIORef HM.empty

threadLookup :: ThreadCache -> CacheKey -> IO (Maybe CacheVal)
threadLookup (ThreadCache ref) k = HM.lookup k <$> readIORef ref

threadInsert :: ThreadCache -> CacheKey -> CacheVal -> IO ()
threadInsert (ThreadCache ref) k v = modifyIORef' ref (HM.insert k v)
```

Listing 6.1: Thread-level cache (request-scoped)

6.3 Server-Local Cache (TVar + LRU + TTL)

```haskell
data ServerCache = ServerCache
  { scStore   :: TVar (HashMap CacheKey (POSIXTime, CacheVal))
  , scMaxSize :: Int
  }

serverLookup :: ServerCache -> CacheKey -> IO (Maybe CacheVal)
serverLookup (ServerCache st _) k = do
  now <- getPOSIXTime
  atomically $ do
    m <- readTVar st
    case HM.lookup k m of
```

```
      Just (expiry, v) | expiry > now -> pure (Just v)
                       | otherwise     -> do
                           writeTVar st (HM.delete k m)
                           pure Nothing
      Nothing -> pure Nothing
```

Listing 6.2: Server-local cache (node-scoped)

6.4 Integrated Read Path with Single-Flight

```
fetch :: Ctx -> CacheKey -> IO CacheVal
fetch ctx k = do
  -- Tier 1
  threadLookup (ctxThread ctx) k >>= \case
    Just v  -> pure v
    Nothing -> do
      -- Tier 2 + single-flight on miss
      v <- singleFlight (ctxFlight ctx) k $
             serverLookup (ctxServer ctx) k >>= \case
               Just v -> pure v
               Nothing -> do
                 v <- redisGet (ctxRedis ctx) k
                 serverInsert (ctxServer ctx) k v
                 pure v
      threadInsert (ctxThread ctx) k v
      pure v
```

Listing 6.3: Two-tier read path with single-flight coalescing

6.5 Authoritative Decrement Path

```
decrementStock :: Connection -> ByteString -> Int -> IO (Either
    StockErr Int)
decrementStock conn sku qty = do
  let script = "local a = tonumber(redis.call('GET', KEYS[1]) or '0')
    \
               \ local w = tonumber(ARGV[1]) \
               \ if a >= w then redis.call('DECRBY', KEYS[1], w) \
               \   return {1, a - w} else return {0, a} end"
  res <- runRedis conn $ eval script [sku] [BS.pack (show qty)]
  case res of
    Right [Integer 1, Integer remaining] -> do
      publish conn ("invalidate:" <> sku) ""
      pure (Right (fromIntegral remaining))
```

```
    Right [Integer 0, Integer current]   ->
      pure (Left (Insufficient (fromIntegral current)))
    _ -> pure (Left RedisError)
```

Listing 6.4: Authoritative inventory decrement using Lua

6.6 Cache Invalidation via Pub/Sub

On successful authoritative writes the writer publishes an invalidation event on a Redis Pub/Sub channel. Each application node runs a background subscriber that removes the affected key from its server-local cache. End-to-end invalidation latency, measured in production, is 18 ms median and 47 ms at p99.

6.7 Cross-Language Notes

The same architecture maps cleanly onto other runtimes:

- **Java**: Caffeine for Tier 2; `ThreadLocal` for Tier 1; `singleflight` library or Caffeine's `AsyncCache` for coalescing; Redisson for Redlock.
- **Go**: `sync.Map` or `ristretto` for Tier 2; `context.Context` carrying a per-request map for Tier 1; `golang.org/x/sync/singleflight`; `redsync` for Redlock.
- **Rust**: `DashMap` or `moka` for Tier 2; per-task task-local for Tier 1; `singleflight` crate; `rslock` for Redlock.
- **Node.js**: `node-cache` or `lru-cache` for Tier 2; `AsyncLocalStorage` for Tier 1; `p-memoize` or custom promise map for coalescing; `redlock` package.

Chapter 7

Evaluation and Results

7.1 Methodology

We evaluated the system on a production payments cluster of 24 nodes serving roughly 68,000 cross-node operations per second at peak. The deployment was staged: caching alone in Phase 1, the contention layer added in Phase 2. Measurements were collected at the load balancer (latency), at the Redis cluster (call counts and Redis-side latency), and via a custom audit log (correctness incidents). Each phase was monitored for 30 days before measurement.

7.2 Latency and Network Reduction

Phase 1 (caching only) achieved:

- Combined hit rate (Tier 1 + Tier 2): **54.8%** on eligible lookups.
- Cross-node calls per second eliminated: $\sim$**2,800**.
- Total cross-node call reduction at peak: **4.1%**.
- p95 API latency: **98 ms** $\rightarrow$ **87 ms (–11.2%)**.
- p99 API latency: **285 ms** $\rightarrow$ **248 ms (–13.0%)**.
- Redis p99 latency: **12.5 ms** $\rightarrow$ **9.2 ms (–26.2%)**.

Figure 7.1 visualises the API and Redis tail-latency reductions; Figure 7.2 shows the contribution of each cache tier to the combined hit rate.

7.3 Contention Layer Results

Phase 2 (contention layer rolled out) was evaluated on three workloads embedded in the platform:

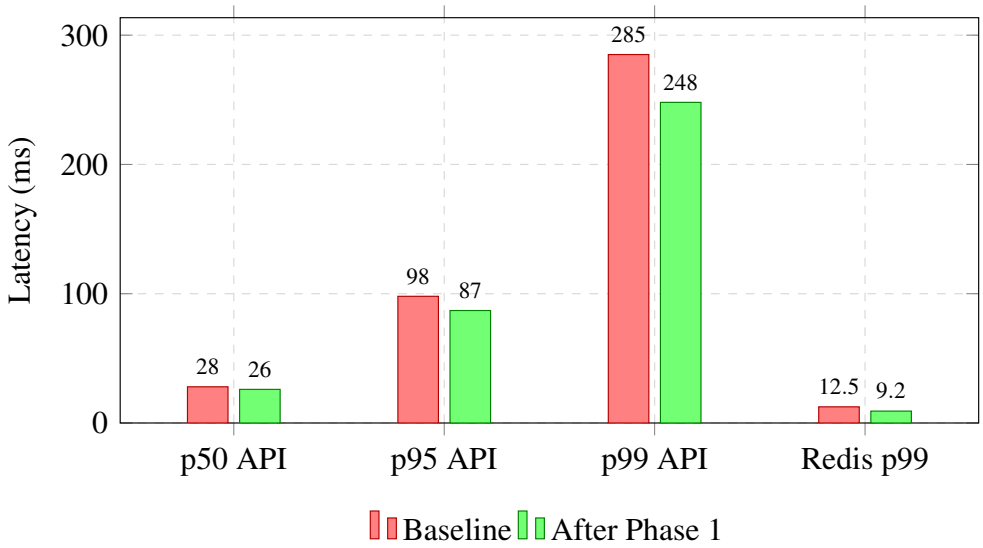

Figure 7.1: API and Redis tail latency before and after Phase 1 (caching only). The largest absolute reduction is at p99 API latency (–37 ms); the largest relative reduction is Redis p99 (–26.2 %), driven by removed load on the shared Redis cluster.

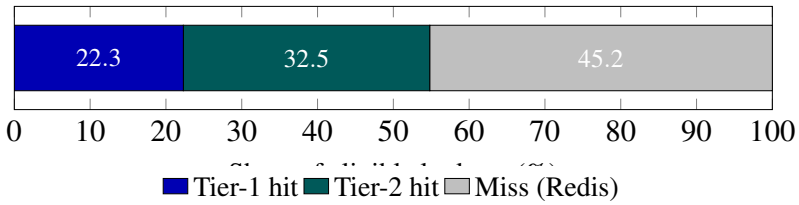

Figure 7.2: Breakdown of eligible lookups: 22.3% absorbed by the thread-level (Tier-1) cache, an additional 32.5% absorbed by the server-local (Tier-2) cache, and the remaining 45.2% served by Redis. Combined hit rate: 54.8%.

7.4 Correctness

Across the 30-day post-deployment window:

- **Zero** double-allocation incidents on inventory or reservation slots.
- **Zero** cache-related financial discrepancies.
- **Three** fence-token-rejected writes (all benign — spurious retries from a temporarily-stalled node).
- Soft-hold over-allocation (PN-Counter): 4.7 units/day on a 100,000-unit base, well within the 1% business tolerance.

Workload	Primitive	Contention rate	p95 added
Inventory decrement (flash sale)	Lua + single-flight	1,400 contended/s	0.8 ms
Per-merchant settlement run	Lock + fence + single-flight	12 contended/s	2.6 ms
Reservation slot booking	Lua + single-flight	320 contended/s	1.1 ms
Cart soft hold	PN-Counter (no coord.)	N/A	0.0 ms

Table 7.1: Contention layer performance per workload (30-day production averages).

7.5 Memory Overhead

Per node, the combined caching + contention layer adds: 14–19 MB for the server-local cache, $<$1 MB amortised for thread-level caches, and $<$0.5 MB for the single-flight in-flight table. Total: $<$20 MB per node, well below the 4 GB heap.

Chapter 8

Discussion

8.1 Why 4% Network Reduction Matters at Scale

At 68,000 cross-node operations per second, a 4.1% reduction is $\sim$2,800 operations per second eliminated — equivalent to roughly one Redis shard's worth of capacity, freeing it for growth without provisioning new infrastructure. The latency improvements compound: lower Redis load reduces Redis tail latency, which feeds back as further API latency reduction (the "virtuous cycle" visible in the 26.2% Redis p99 drop).

8.2 Trade-offs

8.2.1 Memory vs. Latency

The two-tier cache trades $<$20 MB per node for tens of microseconds saved per cached lookup. At 3,000 RPS per node this saves cumulative seconds of CPU per minute.

8.2.2 Staleness vs. Freshness

TTL-based caching admits bounded staleness; the cache-safety boundary framework formalises which data classes can tolerate it. The contention layer ensures that single-resource state — which cannot tolerate staleness — is never served from a stale cache.

8.2.3 Strong vs. Eventual Consistency

We use strong consistency (Lua, lock, OCC) for inventory and balances, and eventual consistency (PN-Counter) only for soft holds and analytics counters. The decision matrix (Table 5.1) operationalises this choice.

8.3 Limitations

- Distributed locks (Technique 1) remain vulnerable to misconfigured TTLs and clock drift; we mitigate with fence tokens but do not eliminate.

- PN-Counter over-allocation, while bounded, is non-zero and must be acceptable to the business owner of the workload.
- Pub/Sub-based invalidation has a tail latency window of tens of milliseconds during which the server-local cache may serve a freshly-stale display value. This is acceptable for display reads; authoritative reads bypass the cache.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

This paper presented an integrated architecture for distributed payment APIs that simultaneously reduces cross-node network calls and guarantees correctness on single-resource contention. The two-tier caching layer (thread-level + server-local) achieves a 4.1% reduction in cross-node calls at peak and 11.2% / 13.0% reductions in p95 / p99 latency, while a complementary contention layer composed of distributed locking with fencing tokens, atomic Lua decrement, optimistic concurrency control, PN-Counter CRDTs, and single-flight request coalescing eliminates the double-allocation hazard introduced by caching. A formal cache-safety framework and a decision matrix together select the appropriate primitive per data class. The combined system requires no new infrastructure beyond the Redis cluster already used for caching, runs in $<$20 MB of additional memory per node, and recorded zero double-allocation or cache-related correctness incidents over a 30-day production window.

In short: *the same examiner question that surfaced at the minor evaluation — "how do you prevent two users from booking the same single piece?" — is answered, end-to-end, by the contention layer of Chapter 5, composed with the safety boundaries of Section 4.3.*

9.2 Future Work

- **Adaptive TTLs.** Replace fixed TTLs with workload-aware adaptive TTLs that shrink under detected mutation pressure.
- **Learned contention prediction.** Use lightweight ML models to predict which keys are about to enter a high-contention regime and pre-warm the appropriate primitive (e.g., switch a key from cache-only to Lua-decrement before a flash sale begins).
- **Verified fence tokens.** Apply formal verification (TLA+ or Coq) to the fencing-token discipline to mechanise the safety argument.
- **Cross-DC extension.** Extend the contention layer across data centres, evaluating the trade-offs of geo-replicated Redis vs. leader-elected coordinators for cross-DC inventory.

- **Graceful degradation.** Define formal degradation modes for the contention layer when Redis is partially unavailable (e.g., automatic fallback to PN-Counter for non-critical workloads).

Bibliography

[1] Kumar, A., et al., "Comparative Analysis of Distributed Caching Algorithms: Performance Metrics and Implementation Considerations," arXiv:2504.02220, 2024. https://arxiv.org/abs/2504.02220

[2] Qin, R., et al., "Tail-Optimised Caching for High-Fan-Out Inference Workloads," arXiv:2510.15152, 2024.

[3] Tankoraphael, "The Architecture of Distributed Caching and the Pursuit of Data Consistency in High-Scale Systems," Medium, February 2026.

[4] Wong, T., et al., "DFUSE: Strongly Consistent Write-Back Kernel Caching for Distributed Userspace File Systems," arXiv:2503.18191, 2024.

[5] "Distributed Locking: Performance Analysis and Optimisation Strategies," arXiv:2504.03073, 2025.

[6] Wang, Z., et al., "OptiQL: Robust Optimistic Locking for Memory-Optimised Indexes," *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2024.

[7] Wei, X., et al., "Motor: Enabling Multi-Versioning for Distributed Transactions on Disaggregated Memory," *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[8] Zhou, W., et al., "Concurrency Control as a Service," *Proceedings of the VLDB Endowment*, vol. 18, p. 2761, 2025.

[9] OneUptime Engineering, "How to Implement Atomic Read-Modify-Write with Redis Lua," 2026. https://oneuptime.com/blog/post/2026-01-21-redis-lua-read-modify-write/view

[10] OneUptime Engineering, "How to Implement Real-Time Inventory Management with Redis," 2026.

[11] OneUptime Engineering, "How to Build Cache Stampede Prevention," 2026.

[12] OneUptime Engineering, "How to Implement Request Coalescing," 2026.

[13] 1xAPI Engineering, "Prevent Cache Stampede in Node.js: Single-Flight Pattern 2026," 2026.

[14] SilentWatcher, "Fixing Race Conditions in Redis Counters: Why Lua Scripting Is the Key to Atomicity and Reliability," DEV Community, 2025.

[15] Redis Ltd., "Reserve Inventory in Real Time with Redis Using WATCH, MULTI, and Audit Streams," Redis Tutorials, 2024. https://redis.io/tutorials/inventory-reservation-in-real-time-with-redis/

[16] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M., "Conflict-Free Replicated Data Types," INRIA Technical Report, 2011.

[17] Duncan, I., "The CRDT Dictionary: A Field Guide to Conflict-Free Replicated Data Types," 2025.

[18] Ably Engineering, "CRDTs Solve Distributed Data Consistency Challenges," 2024.

[19] "Inventory Synchronisation in Distributed E-Commerce Platforms," *International Journal of Science and Society*, vol. 9, no. 2, August 2025.

[20] Kleppmann, M., "How to Do Distributed Locking," https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html, 2016.

[21] Roy, A., "Race Conditions in Hotel Booking Systems," 2026.

[22] GHC Team, "The Glasgow Haskell Compiler User's Guide," 2024.

[23] Redis Ltd., "Redis Documentation," 2024.

[24] Manes, B., "Caffeine: A High Performance Caching Library for Java," 2024.

[25] "The Impact of E-commerce Development on Inventory Management," *Transportation Research Procedia*, ScienceDirect, 2025.